

loggy: a logging platform for the ALICE Event Processing Nodes

Keep the volume where it is made, and the value where it is safe.

Marko Sladojevic

CERN, ALICE Event Processing Nodes

Supervisors: Lubos Krcal, Federico Ronchetti

September 2026

Abstract

The ALICE Event Processing Nodes write logs that mostly never reach the tool people read with. loggy keeps them on the worker that wrote them and searches every worker as one. One collector per worker reads six log sources and stamps each line with a template. Info, debug and trace stay in one local copy. Every other record goes to three storage machines. Every worker pushes its health up. Nothing polls it. Threshold rules, Random Cut Forest detectors and trend rules turn numbers into episodes. It runs on five staging machines. A farm deployment of the cluster, the collectors and Dashboards passed every check. In the busiest hour of six months of archive the median worker carried 23 records a second, the busiest worker-second 78. The collector costs about a quarter of one core at 20,000 records a second. The whole stack on one worker sustains about 42,000 a second. A flush interval of 1 second instead of 5 cut the collector's cost 24.5 % and its peak memory 28 %. The templating pipeline is 31 to 57 % cheaper per family and byte-identical on 8,961,245 lines. Dense retrieval with reranking reaches 0.685 nDCG@10 against 0.634 for the scan alone. The main limit: every cost figure comes from a laptop. Only the shape of a curve and the ranking of settings transfer to the farm.

Contents

1	Introduction	1
2	The problem and the constraints	1
3	The design	3
4	Why these choices	10
5	What the measurements say	11
6	Anomaly detection on the log text	13
7	Anomaly detection on the metrics	17
8	Limits and what remains	21
9	Conclusion	23
A	Upstream roles weighed and not used	23

1 Introduction

Most logs an Event Processing Node writes never reach the one tool people use to read them. An Event Processing Node, an EPN, is one machine of the ALICE online farm. Its job is timeframe reconstruction: it assembles the data of one time slice from the experiment into events. The reconstruction software is O2, the ALICE Online-Offline computing system [3]. Log search must not compete with that job.

Logging today is InfoLogger [2]. A client library sends each process's messages to a local daemon, then to one central server. That road has three limits: one table for every message, no counts over time, and one unreplicated retention rule. The process logs on disk and the journal, the log of every system service, never take it (Figure 1).

We built loggy. It keeps the bulk of the logs on the worker, the EPN that wrote them, and still searches them as one. It runs on five staging machines, and one farm deployment of the cluster, the collectors and Dashboards passed every check.

Section 2 gives the constraints. Section 3 walks an error line, an info line, a dead collector and a quiet worker through it. Section 4 names the fact behind each choice, Section 5 what the measurements settle. Section 6 covers anomaly detection on the log text and Section 7 on the metrics. Section 8 gives the limits and what remains.

2 The problem and the constraints

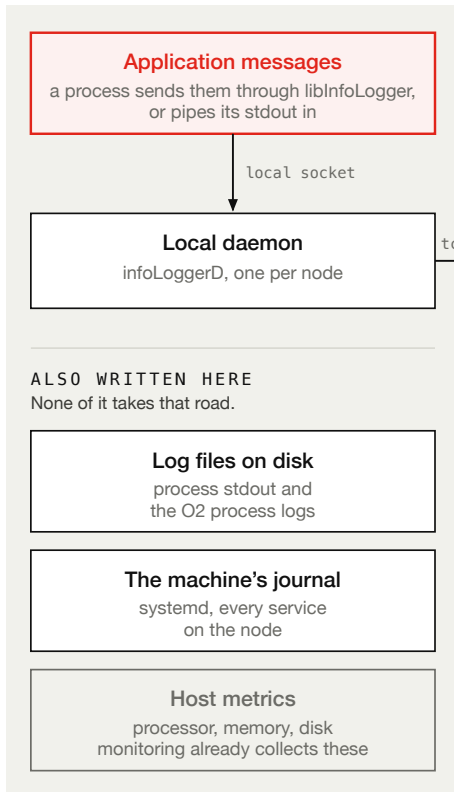
2.1 What a node writes

A worker holds five log families, and the InfoLogger records are a sixth source (Table 1). DDS, the Dynamic Deployment System, starts the reconstruction processes. The journal holds 2,500 to 10,500 entries a day.

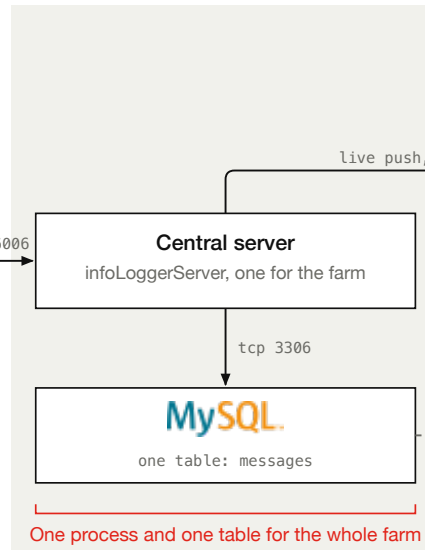
The survey of the real machines set two rules. First, the run directory sits on one 14 TB scratch disk, 93 % full, that every machine mounts over the network. Each collector tails only its own machine's directory. A tail over the whole disk lands every file on every machine. The mount raises no change events, so the tail polls. Second, one O2 process prints raw terminal colour codes into its log, so every anchored pattern fails without a word.

The stored logs we replay are recorded farm logs, fed back through the collector at a chosen rate. They hold InfoLogger, DDS and the process tree, and no journal. The parser corpus is 45,596,613 of those lines from

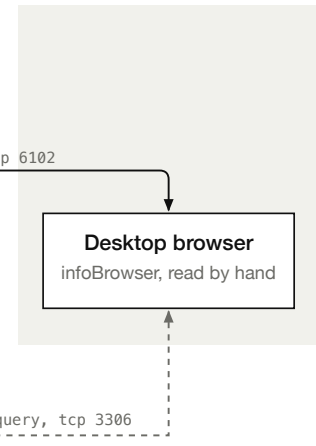
01 ONE EPN WORKER



02 CENTRAL, ONE FOR THE FARM



03 A DESKTOP



04 THE LIMITS TODAY

- **ONE SQL TABLE**
Every message in the farm lands in one messages table.
- **NO AGGREGATION**
infoBrowser lists single lines. It draws no counts over time.
- **ONE POLICY, ONE COPY**
One retention rule covers every message. Nothing is replicated.

Figure 1: InfoLogger carries only what a process sends it, and the process logs on disk and the journal never take that road.

Table 1: Only the InfoLogger records take the InfoLogger road, and the run orchestrator log reaches no collector.

Family	What writes it	How it reaches the collector	Tier it lands on
InfoLogger	Any process, through the client library and local daemon	A TCP socket	Storage, every record
DDS	The deployment system, one file a day in the run directory	Tail of the file	By severity
O2 process tree	Every reconstruction process, one output and one error file each, in two line formats	Tail of the files	By severity
InfoLogger daemon log	The local daemon, one file that never rotates	Tail of the file	Storage, every line
Journal	systemd, every service and the kernel ring	The journal reader	By severity
Run orchestrator log	The orchestrator, on the shared infra machine only	Not collected	None

186 programs. DDS is 0.1 % of that corpus, and operations call it the largest family during data-taking. No run was active during either survey, so no live job log was seen.

2.2 What a worker carries

A real worker carries tens of InfoLogger records a second, not thousands. The InfoLogger archive, six months of dumps, holds 248,828,513 records from 312 hosts. In the busiest hour the median worker carried 23 records a second, the busiest worker-second 78, and the farm 9,781. These figures are floors: the client library cuts off a process at 1,000 messages a minute, so the loudest processes are hidden.

The soak, the project's measurement campaign of twenty-one rounds, tested one collector at 1,000, 20,000 and 50,000 records a second per worker. Those rates are a safety margin, not an archive figure.

2.3 The constraints

A worker gives the platform four of its 64 physical cores, eight of its 128 logical processors, and little memory. No bulk crosses the wire: each worker indexes its info-severity lines on its own disk. Physicists' staging runs occupy two of the three farm workers and a build pipeline wipes the third, so hard memory and processor caps apply.

The infra machine's default ports belong to a second search cluster, so the storage nodes and Alertmanager take other ports. CERN hosts take no long-running agent, so we push over SSH. Two of the three workers cannot install a collector newer than release 4.0.1, because newer releases need OpenSSL 3.4.0 and their system ships OpenSSL 3.2.2. The third worker runs a newer release, so every deployed release must accept every configuration.

2.4 What the platform must do

Six objectives keep the volume where it is made and the value where it is safe (Table 2).

Table 2: One named element of the design meets each of the six objectives.

Objective	Design element that meets it
No bulk over the wire	A local index on every worker, written by a collector that talks only to the search node on its own machine
Split by severity	The collector routes each line by severity to the local index or the storage tier
Distributed	One cluster of two tiers, and every worker answers a search over its own index
Durable	Three machines, each holding a copy of every record above info and of every InfoLogger record
Workers stay cheap	Four physical cores, no cluster management on a worker, and health numbers pushed up rather than polled down
Fault tolerant	The cluster runs while two of the three storage machines agree, and a lost worker loses only its own local index

2.5 The 2025 reference design

loggy kept the shape of the 2025 reference design and changed its rules [1]. We kept its collector on every node and widened it from two log families to five. Its one cluster of two tiers, three index families, lifecycle policies and Alertmanager stayed. We changed its tier rule from source to severity, because a source rule ships one program's bulk and drops another's errors. We replaced Grafana with OpenSearch Dashboards, the store's own web interface, because detection and alerting come with the store [5]. We dropped its queue on the numbers in Section 5.3, and the aggregator it fed. The collectors feed the live lane, the second copy of every record that leaves a worker, directly. We dropped Kubernetes for native system services.

3 The design

loggy is one cluster, split in two by severity.

3.1 One cluster, two tiers

The worker tier runs on every EPN in the target design. The collector reads the six log sources. The stamper beside it names every line's template, the skeleton of the line with every variable token a wildcard, before indexing. The health sample pushes the collector's heartbeat up. The local OpenSearch data node holds one index per worker. The four share the worker's four-core budget of Section 2.3. Memory is a 384 MB cap on the collector and a 1 GB heap on the local node.

The storage tier is three replicated nodes, each a cluster manager and a data node. An index is split into shards, and a replica is a copy of a shard on another node. Two of the three nodes must agree, so the tier survives the loss of one.

The three storage nodes are also the control host, the background host and the shifter host. The control host holds the one web door every person opens. Behind it sit Dashboards, the maintainer's screen, and

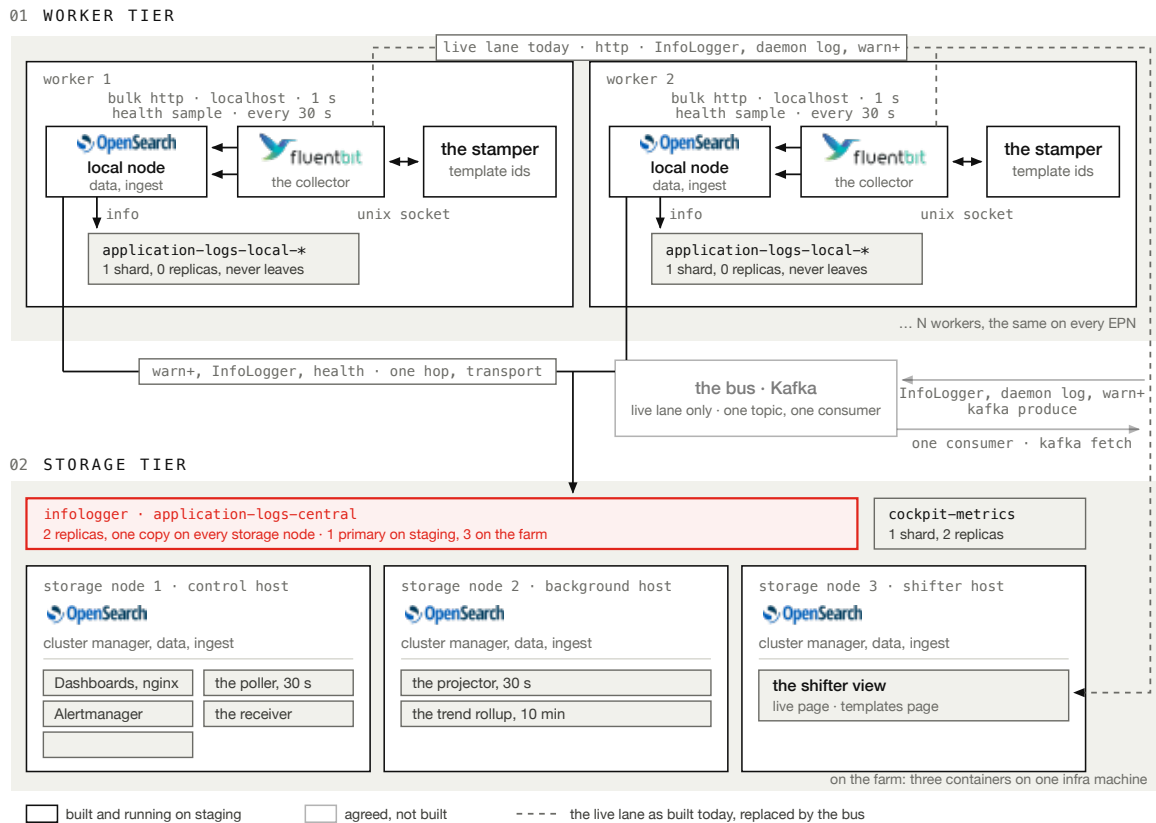


Figure 2: The bulk stays on the worker tier, InfoLogger and every line above info cross once to the storage tier, and every watcher pushes its numbers up.

Alertmanager, which decides when a person is told. The control host also runs the poller, the roster of collectors that must run, the catalogue maintenance and the receiver. The background host runs the projector, which turns alerts into episodes, and the trend rollout. The shifter host serves the shifter view, the page the person on shift watches.

Staging is five machines: two workers and three storage nodes. The farm pilot is three workers and three containers on one infra machine.

The dashed box in Figure 2 is a Kafka bus, agreed for the live lane only. The live lane is the second copy of every record that leaves the worker. That is every InfoLogger record at any severity, the daemon log and every line above info severity. With a bus, moving the shifter view moves one consumer, not more than 200 collector configurations. The bus is not built, and its brokers are not placed.

Table 3 lists the index families. The farm sets three primaries, one per storage node, because one primary funnels all InfoLogger through one machine. No farm run has carried real volume.

Table 3: The bulk sits in one unreplicated index per worker, and every other family sits replicated on the storage tier.

Index	What lands there	Tier	Shards and replicas	Retention
application-logs-local- <node>	info, debug and trace lines, one index per worker	worker	1 shard, 0 replicas	8 days
application-logs- central	warnings, errors, fatals, the daemon log, unparsed lines	storage	staging 1 primary, farm 3, 2 replicas	35 days
infologger	every InfoLogger record	storage	staging 1 primary, farm 3, 2 replicas	56 days
template-buckets-5m, -1h	counts per template version per worker and window	storage	1 shard, 1 replica	4 and 66 days
cockpit-metrics	health samples from collectors and the poller	storage	1 shard, 2 replicas	7 days by document

3.2 Walkthrough: one error line, from an EPN to a screen

An error line crosses the wire twice (Figure 3).

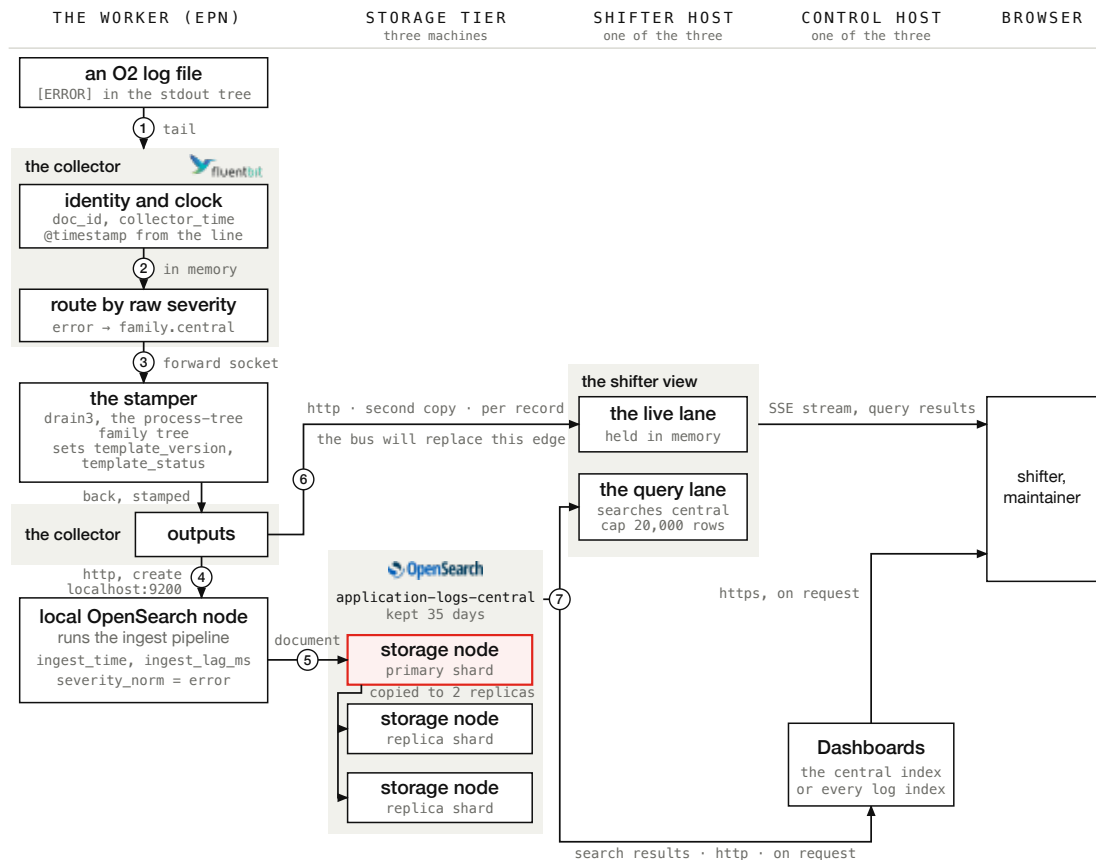


Figure 3: An error line is stamped on the worker, crosses the wire to the storage primary and two replicas, and again to the live lane.

1. The collector tails the file an O2 process writes and reads machine, program and event time. It adds a document identifier and its own accept time.
2. A routing rule reads the raw severity: info, debug and trace go local, an error goes central.
3. Each chunk, the block of records moved as one unit, crosses a local socket to the stamper and returns with template version and status.
4. The local node's ingest pipeline, a set of processors that touch every document before storage, stamps the ingest time and normalises the severity.
5. The document hops once to the storage node holding the primary shard of application-logs-central, which copies it to two replicas.
6. The collector also posts it over HTTP to the live lane on the shifter host. That lane keeps the last 500 records in memory and streams them to every open browser.
7. Dashboards and the shifter view's query lane, capped at 20,000 rows, read the line from the central index.

3.3 Walkthrough: an info line stays on its worker

An info line takes the same road as far as the routing rule, then stays on its worker. A placement setting pins the index there. Under severity routing 96.94 % of the O2 process-tree family stays local and 3.06 % crosses the wire. The trend rollup, six of the seventeen detectors of Section 3.6, and the shifter view's line sampler read the local index. If the worker dies, its raw lines die with it, but the rollup rows, template counts and detector results survive. One search still finds the line. That search is as slow as the slowest worker, which is why the live lane exists.

3.4 Inside the collector

The collector stores nothing (Figure 4).

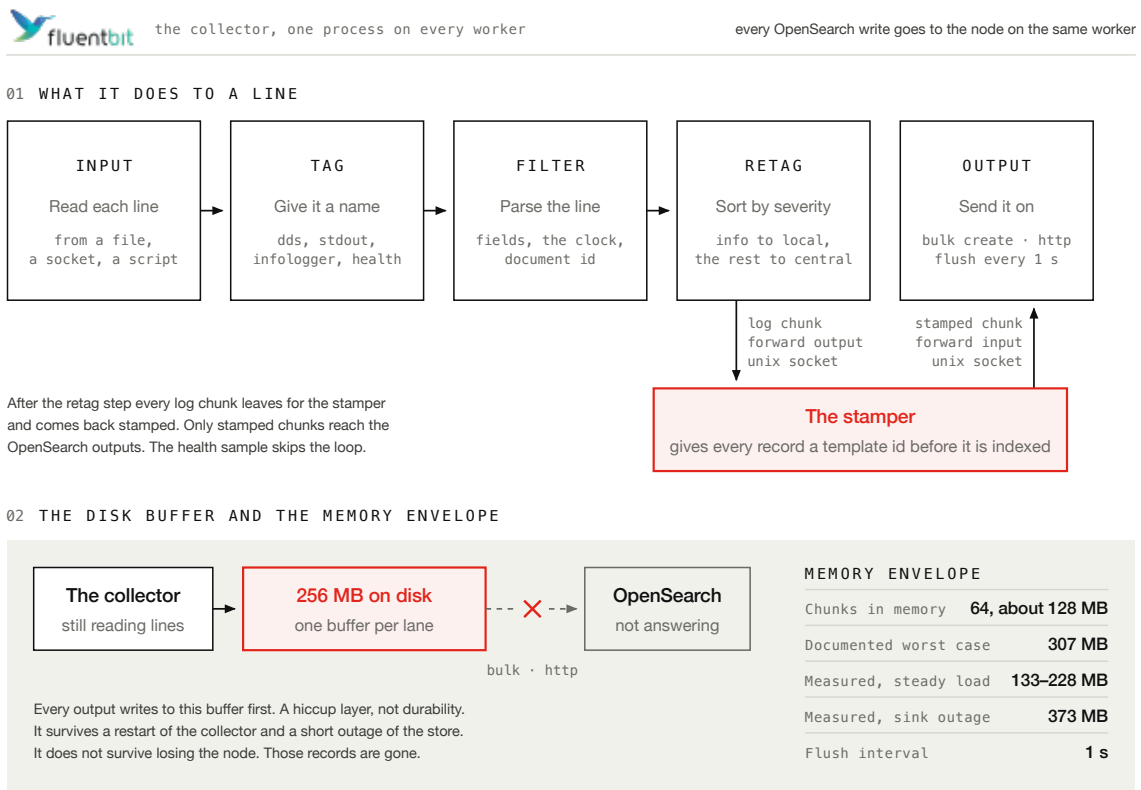


Figure 4: A line takes five steps inside the collector. The 64 chunks it holds in memory are about 128 MB of the 133 to 228 MB it uses under load.

Flushing every second instead of every 5 seconds cut the collector’s processor cost by 24.5 % and its peak memory by 28.0 % (Section 5.2).

At most 64 chunks sit in memory, about 128 MB. The documented worst case is twice that with a 20 % margin, roughly 307 MB.

An output is the collector’s connection to one destination. Each output has a 256 MB disk buffer, a hiccup layer: when the store does not answer, chunks spill to disk. At a real worker’s rate it holds hours, but it is not durability: a lost machine loses its records.

A resend cannot duplicate. The document identifier, node id plus boot id plus a counter, is the record’s id in the store. The output writes with create, which the store refuses when that id already exists.

3.5 Templates in band

Every record carries its template version before it is indexed (Figure 5). A template widens as the stamper sees more lines, and each state is one version.

The stamper runs drain3, a template miner [7]. A masker replaces varying tokens with placeholders, then one tree per family turns a literal that varies between lines into a wildcard. Every record leaves with a status: matched, new or no template. An empty message, or one over 4,096 characters, is indexed in full with no template.

The stamper holds at most 20,000 templates per worker, 4.7 times the 4,221 templates one tree mined from 55,963,050 archive lines (Section 5.5). The shipped service evicts the least recently used template at that ceiling. The soak showed that an evicted template rebuilt later resets its published count to one. The recommended change, to stop learning and keep counting, is open.

The catalogue is one index with one document per template version the farm has seen, plus the results of the checks below. Every 300 seconds the stamper publishes one bucket document per family and window:

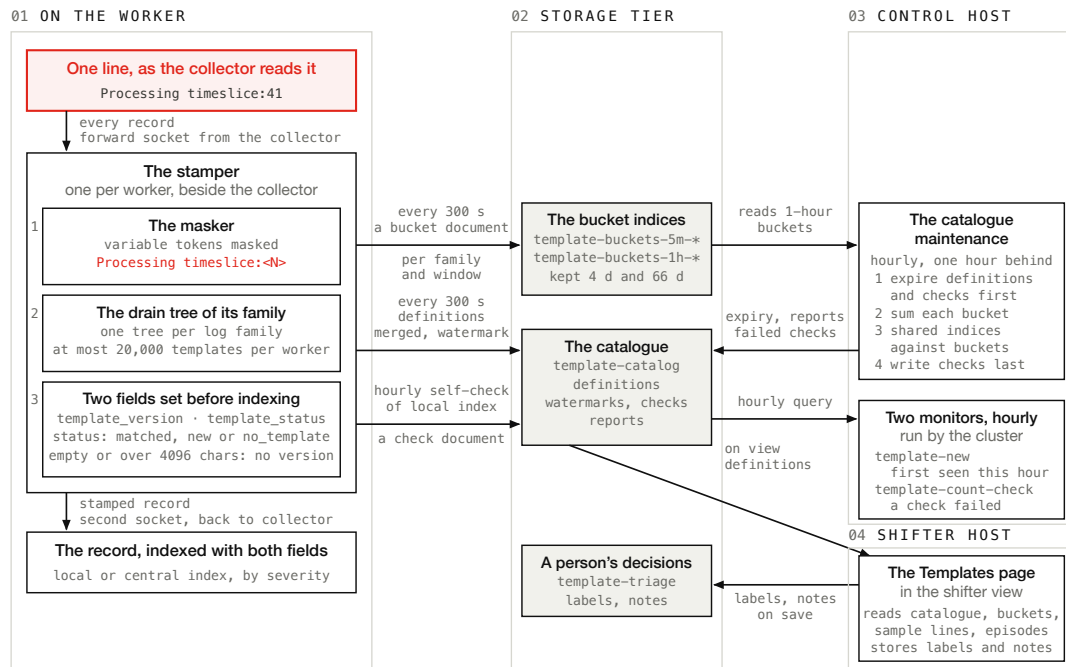


Figure 5: The stamper names each line on the worker, publishes counts every five minutes, and two hourly audits prove the counts add up.

a total and a count per version.

Two audits run hourly. The stamper recounts the previous hour locally. The first check proves that no count is lost: up to 5,000 hourly buckets sum to their family totals. The second proves that no record is counted twice. Neither the central nor the InfoLogger index holds more records of a template version than the workers stamped.

Two hourly rules read the catalogue. One fires on a template catalogued in the last hour. The other fires on any failed check in the last two hours.

A search for a template must also find its older, narrower versions. Within one family and token count, a wider version covers a narrower one when, at every position, it holds a wildcard or the narrower's token. That relation groups the versions, so no separate identifier is stored. Templating is built for search and the catalogue. Two choices are open: catalogue maintenance as store policies plus monitors, and the bucket indices as a rollover alias.

3.6 Watching the platform

Everything that watches the platform is pushed up, never polled down. Every collector ships a health sample every 30 seconds. The poller samples the cluster at the same cadence and reads the roster of collectors that must run: the fleet.

Three lanes read the numbers: threshold monitors catch the cliffs we know, detectors catch unfamiliar shapes, and trend monitors catch slow drift. The numbers are health samples, and counts, error rates and lags from the log records.

A monitor is a saved rule the cluster runs on a schedule, writing an alert when its condition holds. An alert carries one of two tiers. A page wakes a person. A warn waits for one. A storage disk above 92 % raises a page, and above 85 up to 92 % raises a warn. An entity is one machine, or one log family on one machine: the thing a rule is about.

A detector is a scheduled Random Cut Forest model [8]. It learns a number's usual shape and grades each window from 0 to 1 by how unusual it is. Seventeen run, one model per entity. A volume detector reads a silent host as zero records, so it can grade silence. One monitor reads the grades and raises one page for the fleet when any passes 0.7. One forecaster, on disk fill only, trains on a week of hourly points and predicts a day ahead.

Every lane reads a moving window, not the whole history. Each check looks at the most recent stretch of time and compares it with what came just before, so normal moves with the farm. Table 4 gives each window. A detector keeps no fixed number of days. It holds a rolling sample of recent points and forgets older ones gradually, so a new level of traffic becomes normal after a while. The length of that memory follows from the detection plugin’s defaults, which we neither changed nor measured. The trend rules cover what a detector forgets: they compare against a fixed seven days, so a slow drift that a detector absorbs still fires.

Table 4: Every lane compares a recent window with a recent past, so what counts as normal moves with the farm.

Lane	One window	Compared with
Threshold monitors	checked every minute or every ten minutes	a fixed limit
Fast detectors, 10	one minute; each point joins the last 8 windows	a rolling sample of recent points
Slow detectors, 7	30 minutes; each point joins the last 8, four hours	a rolling sample of recent points
Trend rules, 9	three ten-minute slices, ending ten minutes before the check	the same entity over the previous seven days
Forecaster, 1	one hour	the last 168 hourly points, seven days; predicts 24 hours ahead
New-template rule	the last hour	every template the catalogue has seen
Counts per template	300 seconds	not read yet

Thirty monitors run: 13 every minute and four every ten minutes on thresholds and detector results. Nine are trend comparisons, two run hourly on templates, two watch the projector and Alertmanager themselves. The two template rules are the detection on the log text today (Section 6).

The projector reads every alert and detector result every 30 seconds (Figure 6). A signal is one stored row: which rule or detector fired, on which entity, when. An episode is one trouble on one thing, with a start and an end. The projector stores each episode as one incident document, which groups the signals of one alert name on one entity. An episode closes after consecutive healthy windows: one for a threshold monitor, three for a trend monitor or detector. A grade above 0.5 opens an episode.

Alertmanager owns notification semantics only: grouping timers, silences and routing [9]. It forgets an alert five minutes after the last post, so the projector re-posts every open episode every 30 seconds. It holds a page 30 seconds and a warn five minutes, then sends one batch to the receiver. Nothing pages a human today.

The two monitors that watch the projector and Alertmanager post straight to the receiver, because that part can be dead. That is the break-glass path. The stale-projector monitor fires after ten minutes without a projector heartbeat. The other fires after the projector reports Alertmanager down for five minutes.

3.7 Walkthrough: a dead collector becomes a notification

A dead collector is silent, so the platform reasons from absence. Every 30 seconds the poller writes one row per rostered collector, and flags the collector silent for 90 seconds. The collector-down monitor reads those rows every minute and fires at the page tier. The projector opens an episode and posts it to Alertmanager. Alertmanager groups on alert name and collector and posts one batch after 30 seconds.

The cadences bound death-to-document at 90 to 270 seconds. The first missing flag arrives 60 to 150 seconds after the death. No run has timed it (Section 8.1).

3.8 Walkthrough: an EPN goes quiet, or loud

A rollout turns raw lines into small rows: one per family and host per ten-minute slice. It runs every ten minutes, waits 120 seconds for late records, then recomputes the last three finished slices. Each row holds record, error and fatal counts and the 95th percentile of two lags. The lags are the line’s age when the collector read it, and its trip from there into the store. A host that logged in the last 24 hours but not this slice gets a zero-count row. A host silent for longer gets none.

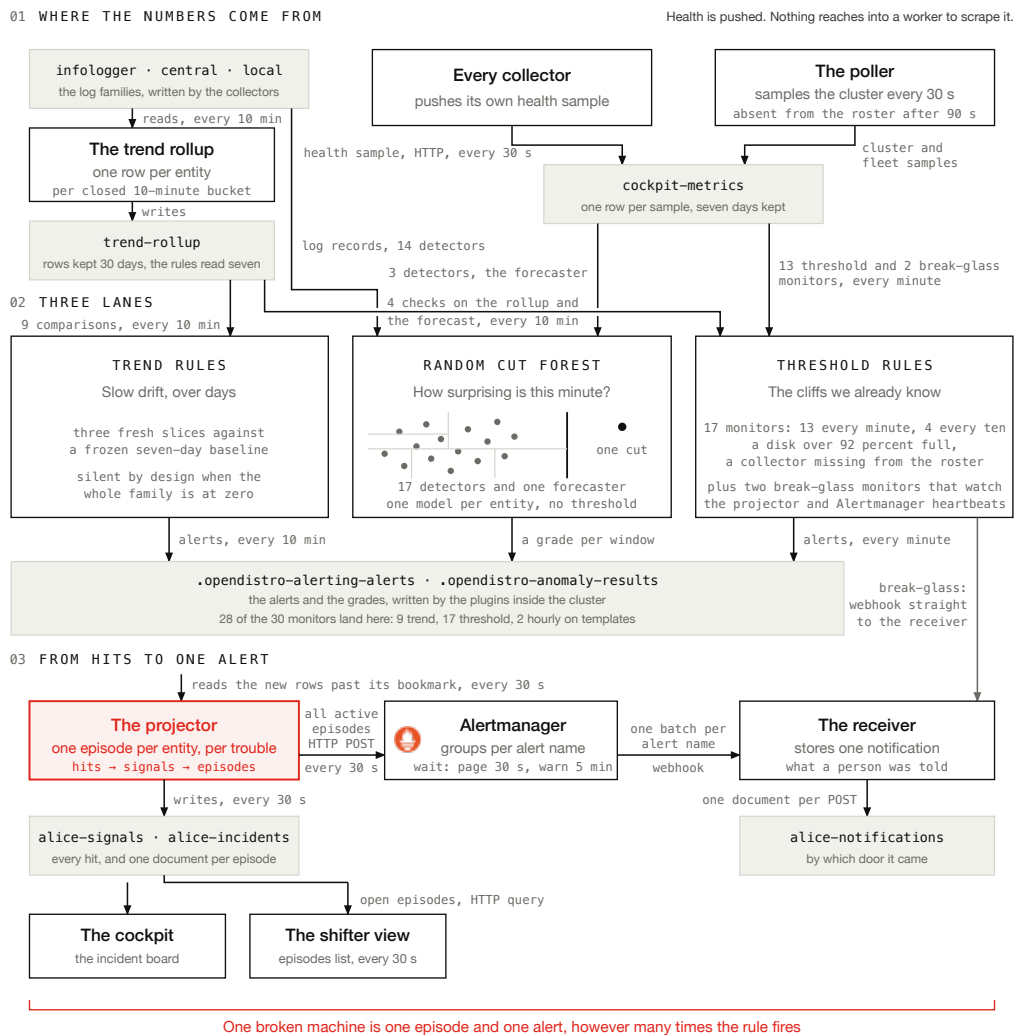


Figure 6: Pushed samples feed three lanes, the projector makes episodes, Alertmanager decides when a person is told, and the receiver stores what was sent.

The nine trend monitors read those rows only. Each compares three ten-minute slices, ending ten minutes before the check, against the previous seven days. A volume rule compares the entity's share of the fleet, so a fleet-wide ramp cancels. It fires on twice the baseline share or half of it, in all three slices. It needs 50 records per slice and six baseline slices.

The two shipping-lag rules compare the entity's own lag against twice its seven-day baseline, floored at 250 ms. A separate monitor raises a page when a family averaging at least 50 records a slice over 24 hours writes nothing for 40 minutes. Trend alerts are the warn tier, so Alertmanager batches per rule name. Which messages changed is the template lane's question (Section 3.5).

3.9 The surfaces

Everything a person opens sits behind one door: one address, one account, four surfaces. Dashboards, the store's own web interface, holds the maintainer cockpit and Discover, its search screen for the archive [5]. The shifter view is separate. Its live page keeps what collectors post: every InfoLogger record, the daemon log and every line above info severity. It never queries the cluster, so it holds when the cluster is down. Its Templates page shows the catalogue, counts and open episodes.

Two clocks run. Health samples and the log-counting detectors use the collector's accept time. Discover uses the time the line was written. The operator view close to InfoLogger is not built.

4 Why these choices

The collector. Maturity and size decided it. Fluent Bit [4] is graduated at the Cloud Native Computing Foundation, its top maturity level [13]. Table 5 lists the four.

Table 5: Fluent Bit won on maturity and size.

Candidate	Language	Memory at rest	Governed by	Verdict
Fluent Bit	C	about 450 KB idle	CNCF, graduated	chosen
Fluentd	C and Ruby	over 60 MB	same project	Ruby tree on every worker
Vector	Rust	not published	Datadog	one company decides it
Telegraf	Go	not published	InfluxData	a metric, not a log line

The store. An open licence was the floor, and detection decided it. Only OpenSearch [5] indexes the message body, counts over time and ships detection under an open licence. Table 6 lists the four.

Table 6: OpenSearch is the only open store that ships detection.

Candidate	Licence	What it answers	Governed by	Verdict
OpenSearch	Apache 2.0	body, counts, detection and alerting	Linux Foundation	chosen
Elasticsearch	three licences	body and counts, detection paid	one company	detection is a purchase
ClickHouse	Apache 2.0	counts on disk, no detection	one company	detection is ours to build and run
Loki	AGPLv3	labels, not the text	one company	a count is a scan

One cluster. We run one cluster, and Dashboards asks it once. Many clusters joined by cross-cluster search, one query forwarded to all, do not scale to 100 machines. The cost is fanout: a search is as slow as the slowest machine. One cluster places its shards itself, keeps one set of users and saved searches, and needs no joining layer. Three hundred data nodes in one cluster, one per worker in the archive, is an untested risk for the cluster manager.

The split by severity. We split by severity, not by source, because info is the bulk and the trash and stays local. A heap holds roughly 20 shards per gigabyte, so three 1 GB heaps hold about 60 shards. Staging runs one primary, because three primaries reach about 135 shards at full retention.

Native services, and containers on one machine. Staging runs vendor packages as system services, because a container runtime on OpenStack is a second runtime. The farm gives the storage tier one shared machine, so its three nodes run there as containers. Three replicas on one disk rehearse no fault tolerance.

The surfaces. Dashboards holds the maintainer cockpit and Discover because it ships the detection and alerting screens. We did not compare Dashboards and Grafana on numbers. The shifter view is a standalone page, because a plugin must match the store's exact version.

The stamper in band. The stamper runs beside the collector, so no raw line leaves its machine. It runs drain3 [7], because no collector filter language can host it. Section 6.3 gives the parser comparison and the masker rewrite. What the stamper costs beside a running collector is not measured.

Detection on numbers and templates. Three lanes read numbers about the logs inside the cluster, and Random Cut Forest detectors [8] read a silent host as zero records. Deep sequence models score about 0.23 to 0.27 on F1 once the logs drift from the data they learnt on. F1 weighs faults caught against false alarms, and 1 is perfect. Plain principal component analysis scores about 0.71 on the same logs. Detection on the log text goes through templates, and Section 6 gives every route on the text and why the others were rejected.

Episodes, and Alertmanager as messenger. The projector turns thirty rows about one dead collector into one episode. Alerting alone repeats the alert every minute. Alertmanager [9] stores nothing durable, so the projector re-posts every open episode. Inhibition, where one alert silences another it causes, is built and ships off. All 22 cause-and-effect links between alerts are unproven.

Kafka between the collector and the store. The soak rejected a queue between the collector and the store. Decoupling: one worker's stack sustains about 42,000 records a second, 538 times its busiest second in six months. Durability: only InfoLogger is lost in an outage, and its 256 MB disk buffer covers 17.4 hours of it at a median worker's rate. Cost: three brokers on memory the staging machines lack, and the tier rule splits into two places. Section 5.3 gives the four conditions that reopen it.

Kafka for the live lane. The September 2026 rework agreed a Kafka [11] bus for the live lane only, for decoupling and not durability, and it is not built. Today a moved shifter view means editing more than 200 collectors. It must be Apache Kafka, because CERN requires fully open source software. It must be the primary path, because the collector held at release 4.0.1 has no route for a failed output. Where the brokers run is not decided.

Deployment. The agent decided it. An agent is a program the tool installs and leaves running on every machine. Ansible [10] has none: it pushes over SSH when a person starts a run, and creates the machines too. Puppet wants an agent and a certificate on every machine. That is likely right for the farm. It is wrong for five machines that one person deploys by hand. Salt and Chef also want an agent, and Terraform builds machines but does not configure them.

Cockpit metrics beside Telegraf and Mimir. The poller marks absence against a roster. CERN already runs Telegraf and Mimir for machine metrics, and our cluster and node samples overlap them. Whether that estate takes the samples over is open. The roster and absence logic have no equivalent there, so they stay.

Shared code as one package. Shared Python becomes one versioned package, agreed and not built. About 31,000 lines of Python run the services, and five modules are byte-identical copies across them. Appendix A lists the upstream roles that lost.

5 What the measurements say

The numbers settle what the stack costs a worker, its ceiling, and what templating costs. Every figure comes from a laptop rig. Shapes, knees and rankings transfer to the farm. Absolute rates and core-seconds do not.

5.1 The instrument and its limits

The rig runs the whole stack in containers. The instrument sums every container's processor time into core-seconds, one core busy for one second. Cost is core-seconds per million records.

Two rigs exist, never averaged. The healthy rig tested the collector's settings at 20,000 records a second. The re-run rig, the same laptop slowed, offered 5,000. An arm is one setting under test, and a cell is one arm at one rate.

The noise floor is the spread of three control runs: 1.61 % at 20,000 records a second and 9.48 % at 1,000. Cost per record falls as rate rises: the collector alone costs 75.32 core-seconds per million at 1,000 a second and 11.19 at 20,000.

The rig saturated unnoticed after that first pass, so we retracted every later measurement. A 47-cell re-run left one product change: the flush interval.

5.2 The collector under load

Against a sink that always accepts, the collector's memory sat between 133 and 228 MB on two cores, up to 53,000 records a second. In a sink outage with the shipped 256 MB buffer it peaked at 373 MB, under its 384 MB memory line.

The flush interval is how often the collector pushes buffered records out. Table 7 comes from three alternating runs. Across eight flush values from 0.125 to 10 seconds, whole-stack cost is a U with its floor at 1 second, best to worst 17.9 % apart.

Table 7: Flush 1 second beats the shipped 5 seconds on every measure, at 5,000 records a second on the re-run rig.

Measure	Flush 5 s (shipped)	Flush 1 s (chosen)	Change
Whole stack, core-seconds per million records	108.40	104.67	−3.4 %
Collector alone, core-seconds per million records	36.70	27.70	−24.5 %
Collector peak memory, MB	87.1	62.7	−28.0 %
Live-lane latency floor, s	5	1	

Every threading arm cost more against the 1.61 % floor: a second collector process 7.4 %, a thread per source 11.5 %, and parsing moved into those threads 26.5 %. One single-threaded loop does 89.3 % of the collector’s processor work, so threads elsewhere cannot help it.

The live lane of Section 3.1 is the largest cost any setting adds: 41 to 68 % more collector processor time on the healthy rig.

A fifteen-minute sink outage at 20,000 records a second loses 66.7 % of InfoLogger at the buffer cap. That buffer holds 865,674 InfoLogger records of 310 bytes, and InfoLogger is 60 % of a worker’s mix. At a median worker’s 23 records a second InfoLogger is 13.8, and the buffer covers 17.4 hours. At the busiest worker-second, 78, it is 46.8 and the buffer covers 5.1 hours.

The stack sustains about 42,000 records a second. It holds 50,000 for two minutes with zero loss. The storage tier runs at 86 % of its four cores. The decline starts near six million documents, where index merging cuts throughput, so the ceiling is the storage tier’s.

Overload is safe. At 82 % of the buffer cap the collector absorbs the shortfall and loses nothing. At the cap it discards oldest-first, 1.8 % of records, without error, then drains in 263 seconds.

Two settings make no difference. On the re-run rig a 2 GB heap moved cost 2.1 % against a 6.7 % floor, so 1 GB ships. Core placement between collector and store moved it 2.7 % against a 5.5 % floor. The rig always pinned four exclusive cores, so sharing them with reconstruction is not measured.

5.3 Kafka between the collector and the store, decided on numbers

The soak rejected a queue between the collector and the store on three counts. Decoupling: the stack sustains 538 times the busiest worker-second and 4.3 times the farm’s peak second of 9,781. If the archive under-counts by ten times, the margin is 54 times. Durability: the collector’s buffer holds hours at a real worker’s rate. Cost: three brokers and a second system to run, on memory the machines do not have.

Two rates reopen the question: a sustained worker rate above about 1,000 records a second, or one within 50 times of 42,000. Two needs do too: a second consumer, or an outage longer than the buffer holds. The live-lane bus is separate.

5.4 The other sources and the real node

The collector had never read three of the six log sources. Those are the second process-tree format, written by the data-distribution processes, the InfoLogger daemon log and the system journal. The first fixture run sent 41.8 % of the corpus to the storage tier with no severity: a parser anchor missed a date the replay prepends. Severity now comes out of 99.83 % of process-tree lines, against a 99 % gate. 96.94 % of that family stays on the worker.

A read-only census of four farm machines corrected four facts. Four installations run there, on three releases where we assumed two. The storage machine’s release is two majors behind and untested. A worker’s journal is small beside millions of process-tree lines, so the collector reads all of it.

The fixture and restart checks passed on the three farm releases and on one newer. A later review found the rotation check itself was flaky. Release 5.0.8 loses every byte appended after rotation, in five of five runs, so it is blocked.

An external review retracted every collector regex cost figure. Two identical configurations differ by up to 19.3 % in processor time, so the rig cannot resolve a change below about a fifth. No regex change is worth making on cost.

We fixed 99 defects, 32 in the instrument, the rest in the collector, the stamper and the catalogue. A retrieved bulk write duplicated every record, so the product now writes with create under a stamped document identifier.

5.5 Templating

Templating is built, and the September rewrite of the steps around the miner made it 31 to 57 % cheaper per family (Table 8). Section 6.3 gives why drain3 was chosen, the recipe per family and the masker rewrite.

The template count grows with runs sampled, not lines. One run of each family on 45,596,613 lines gave 3,011, a floor. Sixteen runs and eighty InfoLogger partitions gave 4,092. One tree carried across that corpus and two further archive pulls, 55,963,050 lines, gave 4,221. New templates arrive at 48.9 per million, about one a second at 20,000 records a second.

Splicing the stamped fields into the record's own bytes cut transport 15 %. Transport is roughly a quarter of the collector-to-stamper hop, which fell 6.5 % on InfoLogger and 4.0 % on the process tree. Repeated key names are 32 % of that wire, free on a Unix socket, so we rejected protobuf and gzip unmeasured.

Table 8: Every row is a ratio or a ceiling measured on one host, and the rewrite left the output byte-identical.

Figure	Value	Reference
Pipeline after the rewrite	31 to 57 % cheaper	byte-identical on 8,961,245 lines
Templates, one tree across 55,963,050 lines	4,221	3,011 from one run per family
New templates, steady state	48.9 per million	about one a second at 20,000 records a second
Collector-to-stamper transport after splicing	15 % cheaper	byte-identical
Growth ceiling	20,000	4.7 times the 4,221 mined, 208.1 MB against 512 MB

5.6 The detection chain, proven by injection

Seven scenarios score whether a signal, an incident and a notification followed one thing broken on purpose. A fault agent on each worker kills its collector, and one beside the projector stops only the projector. A poison replay floods the indices with marked fake errors. The stopped-projector run passes only if one stale-projector notification takes the break-glass path of Section 3.6. The scorer checks presence and order, not time, so the bound in Section 3.7 follows from cadences and is not measured.

6 Anomaly detection on the log text

The platform **does detect on the log text, through templates**. The stamper turns every line into a message form, and the detectors watch those forms: a form never seen before, and how often each known form appears. What we rejected is detection on raw lines and on embeddings. Figure 7 shows where each step of the standard route ended.

6.1 The standard route

Anomaly detection on the log text asks one question: is this message unlike every message seen before? The usual answer has three steps.

1. **Templating.** Replace the varying parts of a line with placeholders, so millions of lines collapse into a few thousand forms. Processing timeslice 41 becomes Processing timeslice <NUM>.
2. **Embedding.** Compute a vector for each form that stands for its meaning. Two messages that say the same thing in different words get vectors close together.
3. **Nearest-neighbour search (k-NN).** For a new message, find the closest vectors already seen. If even the nearest one is far away, the message is **semantically new**: it says something no earlier message said.

Each step can run on the worker or on the storage tier, per line or per template. Those two choices decide the cost.

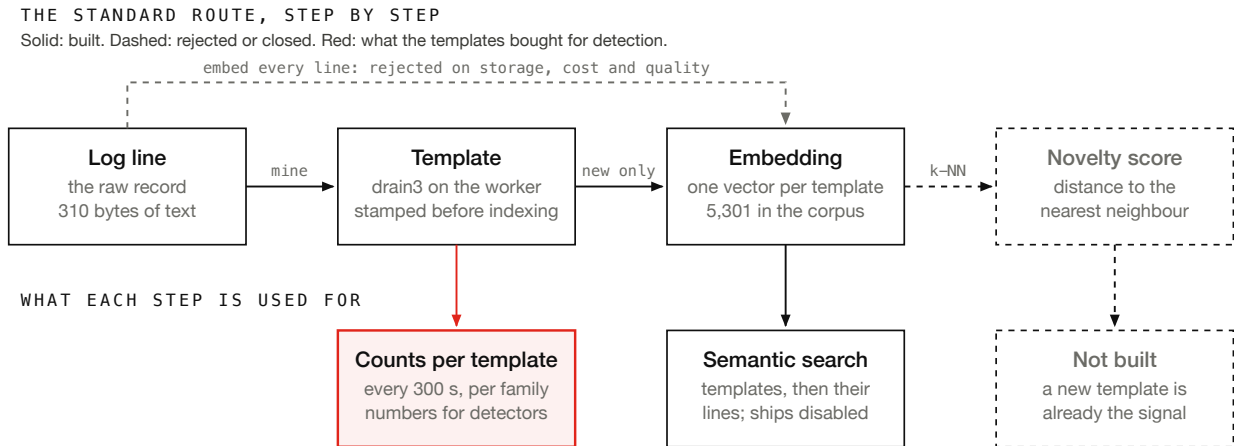


Figure 7: Templates are built and feed counts to the detectors, embeddings serve search only, and the novelty score on vectors is not built.

6.2 Embed templates, not lines

Decision: one vector per template, computed once, never one per line. Table 9 sets the two side by side.

Table 9: A vector per line costs ten times the logs it describes, and the only model fast enough to run per line does not beat plain word search.

	Vector per line	Vector per template
Size of one item	3,072 bytes, ten times the 310-byte record	about 40 kB with its reranking share
Total	about 764 GB for six months of archive, against 77 GB of text	about 211 MB for 5,301 templates
Static model at 23 records a second (median worker)	0.13 % of one core	negligible
Static model at 20,000 records a second (design rate)	1.1 of the worker's four cores	negligible, about one new template a second
The model that wins on quality	113 templates a second on a laptop, so out of reach	47 s for the whole corpus

Storage. Per item a template is dearer, about 129 times a line. Thousands of lines share one template, so the total is what counts.

Worker cost. Encoding every line with a static model costs nothing at a real worker's rate. At the design rate it takes over a quarter of the four logging cores for one feature. Keeping that work off the worker matters at the rate the stack is built for, not at today's rate.

Quality decides it. Only a static model is cheap enough to run per line, and on the sealed set it scores 0.353 against 0.371 for plain word search (Section 6.5). The model that does beat word search is far too slow per line.

What we keep. An operator reaches a line in two steps: **semantic search over templates**, then the store's ordinary search over the lines of the template found. The second step is free, because every record already carries its template version.

6.3 Choosing the template miner

Decision: drain3 [6, 7], one tree and one recipe per log family, behind a rewritten masker. No parser can be scored for accuracy on our logs. The correct template for an O2 line is the format string in the O2 source, and nobody has extracted those. So **cost, memory, streaming and readability** decided. Table 10 gives the reason each candidate lost.

Why one recipe per family. A recipe sets which separators get padded with spaces and how similar two lines must be to share a template. The three families want different answers to the same setting, as

Table 10: Every other parser lost on cost, streaming or readability, and the store's own command works at query time.

Candidate	Why it lost, measured on the same archive lines
drain3	chosen: streams, 26.3 MB, readable once configured per family
PIPLUP	2.56 times the cost and 7.8 times the memory of drain3; one placeholder type for every variable
LogLSHD	slower than the Drain it claims to beat; needs a threshold tuned against ground truth we lack
KELP	crashed three times on real lines, ships no masking, lost on every column once patched
IPLoM, AEL	batch algorithms: they cannot stream
LFA	cheap, but 78.3 % of its templates keep one literal word or none
SHISO, LenMa, Logram	4.8 to 13.3 times the reference Drain's cost
Spell, LogMine	no result in ten minutes
EFParser	calls a language model for every line
The store's patterns command	mines at query time over stored records (reasoning, not measured; see below)

Table 11 shows. One recipe per family costs what one global setting costs, 17.57 against 17.65 weighted core-seconds per million lines. It cuts **contentless templates**, where every token is a placeholder, from 44.9 to 10.4 % on InfoLogger.

Table 11: Each family wants a different separator setting, which is why each gets its own recipe.

Family	Padded separators	Why
InfoLogger	= ; :	full of key: value, so it wants the colon
Process output	= ;	the colon splits real words
DDS	=	a shell command line uses ; between commands

Why the masker was rewritten. The masker replaces numbers, addresses and identifiers with placeholders before the tree sees a line. Without it, mining is **46 times dearer**, because every distinct number becomes a distinct token and the tree grows 92 times the clusters it should have. It was also 74 to 87 % of the mining cost. We rewrote **seven regular expressions** to reach the same output by a shorter path, which made masking 85 to 87 % cheaper with zero differences on 3,000,000 lines per family. The September rewrite of the steps around the miner then made the pipeline **31 to 57 % cheaper per family**, byte-identical on 8,961,245 lines, with the cluster of every line compared in order.

Why not the store's own templating. The store's patterns command mines forms from records already stored, at the moment someone queries. It cannot stamp a template on a record before indexing, it cannot run on the worker where the bulk stays, and it rebuilds its forms on every query instead of keeping one catalogue for the farm. This judgement comes from how the feature works. We did not measure it.

6.4 Templates are the text detector

Templates are how the platform detects on the text. They turn each line into a form, and a form can be counted. Two signals come from them, one built and one half built.

A template never seen before (built). The stamper marks every record as matched, new or without a template, and an hourly rule fires on any template catalogued in the last hour. New templates arrive at 48.9 per million lines in the steady state, about one a second at the design rate.

Counts per template (published, not yet read). Every worker publishes a count for each template version every 300 seconds. That gives a **time series per message form**, which the three lanes of Section 3.6 can read the same way they read a whole family:

- a message that **stops**, when a component has gone quiet;
- a message that **floods**, when one form suddenly dominates its family;
- a message whose rate **drifts** from its own week-old baseline.

Read that way, they would enter the same moving windows as a family's counts (Table 4): a per-template detector would compare each minute with a rolling sample of recent minutes, and a trend rule would compare the last half hour with the previous seven days. No detector or trend rule reads these counts yet. Wiring them in is the next detection step (Section 8.3).

6.5 Embeddings are for search

Decision: DenseOn scans all 5,301 template vectors exactly, and LateOn reranks the top 30. The search is built and evaluated, and it **ships disabled**, because no person has judged its results. The service is capped at 384 MB. The model's memory peak was seen in one local run and is not recorded.

How the benchmark was built. We scored every system on nDCG@10, which rewards relevant templates near the top of the first ten results and reaches 1 when the order is perfect [12].

- **Corpus:** 5,301 template groups mined from 56,628,579 archive lines.
- **Development set:** 97 intent groups and 130 questions, written by two machine writers from real templates.
- **Judging:** four system families pooled their top twenty for 6,410 candidates. Six machine assessors graded every candidate. The lowest weighted agreement between two assessors was 0.761, against a threshold of 0.60 fixed in advance.
- **What counts as real:** 0.0320 nDCG@10, the measured noise between two word-search baselines. A smaller gain is not acted on.
- **Sealed held-out set:** 62 intent groups written before any model was compared, then opened once.

Figure 8 gives the results. The ladder runs cheapest first, from BM25, the store's word-matching score, to LateOn, which scores a question against a template word by word.

DEVELOPMENT SET			
97 intent groups, 130 questions, every candidate graded. The ladder, cheapest first.			
System	What it is	nDCG@10	encode corpus
BM25	the store's word-matching score	0.396	no model
BM25F	BM25 over weighted fields	0.471	no model
potion-base-32M	static model, one vector per word	0.471	—
potion-retrieval-32M	static model, tuned for search	0.479	1.5 s
all-MiniLM-L6-v2	small transformer	0.494	9 s
Qwen3-Embedding-0.6B	large dense model	0.629	1,556 s
mDenseOn	multilingual dense model	0.630	569 s
DenseOn	dense retrieval model	0.640	47 s
LateOn	late interaction, per-word scores	0.695	—

SEALED HELD-OUT SET			
62 intent groups written before any model was compared, opened once. Natural-language questions.			
System	What it is	nDCG@10	
DenseOn	dense retrieval model	0.689	
BM25F	BM25 over weighted fields	0.371	
potion-retrieval-32M	static model, tuned for search	0.353	

THE SHIPPED CONFIGURATION			
One deterministic run: exact cosine scan over all 5,301 vectors, no approximate index.			
System	What it is	nDCG@10	
DenseOn alone	scan, top 10	0.634	
DenseOn + LateOn	scan for 30, rerank 30 in 0.6 ms	0.685	

Figure 8: The dense model beats word search on the sealed set by 0.318, and reranking 30 candidates adds a further 0.051 on development questions.

Why the gap is so large. In 47.1 % of relevant question-and-template pairs, the two share no word after analysis. Operators ask in their own words, and developers wrote the templates in theirs. A dense model closes that gap and word search cannot.

Why DenseOn, then LateOn. LateOn scores best, but searching the whole corpus with it needs a store of about forty vectors per template, and no such store was built or verified. Reranking only DenseOn's top

30 takes 0.6 ms and keeps most of the gain: 0.685 against 0.634, with a bootstrap interval on the gain of +0.028 to +0.080. Bigger models bought nothing. The 0.6-billion-parameter model matched DenseOn and took 33 times as long to encode the corpus. Table 12 lists what else we tried.

Table 12: Every idea tried on top of the dense model either lost or did not clear the bar the project set for itself.

Idea	Effect, nDCG@10	Verdict
Source and program written before the template	+0.168	kept: the largest single gain
Severity and detector added as well	no gain	dropped: 35.4 % of templates carry no severity
Fusing word search with dense results	−0.048	dropped: worse than the dense model alone
Approximate vector index	−0.020	dropped: results changed after each rebuild
Multilingual model	no gain	dropped: the logs are English
Static model distilled with 3,202 ALICE words	+0.039	not promoted: fails the correction for 15 comparisons
The same model further trained on 15,647 ALICE lines	+0.0002	dropped: the vocabulary carries the gain
Static model trained on our own relevance grades	+0.063	a lead only: measured on unseen questions, one split of 43

Post-training a static model. We hoped a cheap static model trained on our logs would retrieve as well as a dense one. Training on log text gave nothing measurable. Adding ALICE words to the vocabulary gave a small gain that did not survive the statistics. Training on relevance grades did generalise to unseen questions, but on one small split, so it stays a lead.

Two limits bind every number here. A machine wrote every question and made every grade, and every latency comes from a laptop.

6.6 No novelty detector on the vectors

Decision: no nearest-neighbour novelty score, and embeddings serve search only. The new-template signal of Section 6.4 already answers the question a novelty score asks, the moment a form is mined, with no vector and no threshold to tune. A distance in the vector space would repeat that signal with a threshold added.

Novelty alone is also a weak signal. O2 software changes across the farm all the time, and a new release prints new forms that are not faults. Detectors that treat every unseen form as an anomaly score 0.23 to 0.27 on F1, where 1 is perfect, once the logs drift from the text they learnt on. Plain principal component analysis scores about 0.71 on the same logs. That is why the counts per template matter more than novelty: a known message that stops or floods is rarely a release. The closest published work on ALICE logs, topic modelling over InfoLogger messages [14], is the comparison for any later text detector.

7 Anomaly detection on the metrics

Two store plugins do the work, and they are not the same thing. A **detector** (Anomaly Detection) grades how surprising an interval is. A **monitor** (Alerting) runs a scheduled search and fires when a condition is true. **A detector writes a grade. A monitor decides whether anyone is paged.** Figure 6 in Section 3.6 shows the whole path, from the numbers to one alert.

7.1 What feeds it

We knew from the start what matters on a worker: **volume**, **errors** and **health**. Two indices hold them.

The log indices give volume and errors per EPN and family. They also give two lags that we compute ourselves. Every record carries three clocks: when the line was written, when the collector read it, and when the store received it. The ingest pipeline subtracts them:

- **Entry lag**, written to read: how late the EPN’s lines reach the collector.
- **Shipping lag**, read to stored: how long the collector takes to deliver.

Both are read as **p95, never the mean**. A backlog shows in the slowest records before it moves the average. **The metrics index** has two writers, and **nothing reaches into a worker to scrape it**:

- **Each collector pushes** its own sample every 30 seconds: alive, records out, errors, retries.
- **The poller** samples the cluster and Dashboards every 30 seconds, plus a roster check: which collectors were expected to speak and did not.

A dead collector writes nothing. So “collector down” is the poller noticing a missing sample after 90 seconds, not a flag the collector sets.

7.2 Detectors: Random Cut Forest

A detector does not read log lines. Every interval it runs a few aggregations, split per entity, and each result becomes a point in a small feature space: volume, error count, p95 lag, heap. A Random Cut Forest [8] then slices that point cloud at random until every point is isolated. **A point in a crowd needs many cuts, a loner needs one or two, and few cuts means a high grade**. Many trees vote, so the grade is stable. The properties that matter here:

- **Streaming**. Normal drifts with the data. A run-start spike alarms briefly, then becomes the new baseline. That is also why a slow leak goes quiet: the model absorbs it.
- **Multivariate**. It flags **combinations that never happened**, such as throughput collapsing while retries climb, or heap rising while indexing falls.
- **One model per entity**. One noisy EPN does not teach the fleet that noise is normal.
- **Shingles**. Each point joins the last eight intervals, so the model reads a short sequence, not one bucket.
- **Window delay**. It waits one minute on metrics and two on logs, so late records still land in their interval.
- **Warm-up**. About 32 live intervals before a detector leaves initialising, and a few hundred before its grades mean much.

Same failure, two horizons. Every log detector runs twice: at **one minute**, where eight intervals give eight minutes of context, and at **30 minutes**, where they give four hours. The slow twin has the same features. It catches a change the fast model has already learnt as normal. Table 13 lists all seventeen.

Table 13: Seventeen detectors: three on health samples, fourteen on the logs as seven fast and slow pairs.

Detector	Per	Watches
<i>Health samples</i>		
ingest-flow	collector	records out, errors, retries; catches a collector going quiet or retrying before a hard rule trips
node-health	store node	heap, processor, indexing rate, disk; catches heap climbing while indexing falls
dashboards-health	whole cluster	event-loop delay, response time, request rate
<i>Logs, per EPN, each with a 30-minute twin</i>		
il-per-epn	EPN	InfoLogger volume and error count: one EPN silent or erroring while the farm looks fine
central-per-epn	EPN	the same on the central family
local-volume	EPN	volume of the info lines that stay on the worker
il-per-epn-entry-lag	EPN	p95 entry lag on InfoLogger
local-per-epn-entry-lag	EPN	p95 entry lag on local info lines
<i>Logs, per collector, each with a 30-minute twin</i>		
il-collector-shipping-lag	collector	p95 shipping lag on InfoLogger
local-collector-shipping-lag	collector	p95 shipping lag on local info lines

Shipping lag is a collector problem, not an EPN problem, so those two are keyed on the collector. One **forecaster**, from the same family, predicts each store node’s disk fill a day ahead from a week of hourly points. Disk is the one metric with a real cliff and a smooth approach.

A detector never pages. It writes a grade from 0 to 1 and a confidence into the results index. The monitor ad-high-grade turns that into an alert when both pass 0.7. A grade is surprise, not harm.

7.3 Monitors: the cliffs we know

A monitor is a **scheduled job inside the cluster**. Every minute, or every ten, the store runs the monitor's search, hands the result to a short script, and fires if the script says yes. Nothing is trained: the script is a threshold, a ratio or a presence test. There are two kinds.

Query-level: one question, one answer. cluster-red asks every minute whether any health sample in the last two minutes says red. The cluster is red or it is not, so one fleet alert is the right shape.

Bucket-level: group, then ask per group. collector-down groups the roster samples by collector and fires once per missing one. If two collectors die, two alerts fire, each naming its machine. A query-level monitor could only say "some collector is down". Throttling follows the same split, so one dead collector does not mute a later page about another.

Table 14 lists the hard rules on the health samples.

Table 14: The hard rules catch failures with a known edge, and each names the machine it is about.

Monitor	Fires when	Means
collector-down	an expected collector sent no sample	that machine, or its collector, is gone
collector-unhealthy	the collector is alive but its health check fails	the box is up, the pipeline inside it is sick
fleet-fb-silence	half the roster is silent at once	one shared failure, so one fleet page instead of a storm
shipping-breaking	a collector retried a write and still failed	the pipe is failing; records may still be buffered
data-loss	a collector discarded records	those lines are gone
cluster-red	cluster health is red	a primary shard has no home
shards-stuck	a shard has had no home for 5 minutes	allocation is blocked
disk-cliff-warn, -page	a store node's disk is 85 to 92 %, or above 92 %	approaching the read-only line near 95 %
heap-spiral	heap above 90 % for 5 minutes	not a brief spike: the node is thrashing
admission-rejections	a node refused a search or a write	overload; a refused write drops logs
telemetry-silence	no cluster or Dashboards sample for 5 minutes	the poller is dead

7.4 Trend rules: the baseline a forest never keeps

Static rules catch cliffs. The forest catches unfamiliar shapes. Neither catches a slow drift, because the forest learns it as normal. The trend lane compares against a fixed week instead.

It rests on our own **trend rollup** service:

1. Every ten minutes it writes **one small row per host and family**: records, errors and fatals, and the p95 and mean of both lags.
2. It waits 120 seconds for late records and rewrites the last three slices. Row identifiers are fixed, so a rewrite never duplicates.
3. A host that logged in the last day and is silent now gets a **zero row**. The store's own rollup writes nothing for silence, which is why this one is ours.
4. A commit row closes each slice, so no rule reads half a slice.

Nine rules read those rows. Each compares **three consecutive ten-minute slices** with the **previous seven days**, leaving out the last 40 minutes so the slices are not part of their own baseline. It fires only when all three slices agree (Table 15).

7.5 Feature engineering was the hard part

The models were the easy choice. Every number fed to them could mislead in its own way:

- **Which machine?** A record names the EPN that wrote it and the collector that shipped it. Keyed on the collector, one model covers a rack and hides a bad EPN. Per-EPN detectors key on the writer.
- **Silence is not zero.** An EPN that stops logging produces no point, and the forest ignores it. Volume detectors fill the gap with zero, so silence becomes the loudest thing they see. Lag detectors do not: a

Table 15: Each trend rule compares a ratio, not a raw count, so a fleet-wide change cancels and one fact raises one alert.

Rule	Compares	Fires when, in all three slices
Volume	the host's share of the fleet	twice the week's share, or half of it; a run start moves every host and cancels
Errors	errors as a share of the host's own records	twice the week's share; doubling traffic and errors together stays quiet
Entry lag	the host's own p95	twice the week's; a slice above one hour is archive age and is ignored
Shipping lag	the collector's own p95	twice the week's; no ceiling, because a long backlog is real

missing lag is unknown, not 0 ms.

- **The mean hides a backlog.** So lag is p95. A p95 over a handful of records is just the maximum, so lag rules need 100 records a slice. The plugin reads only the first percentile it is given, so each lag detector asks for exactly one.
- **A run start moves everything.** Shares of the fleet and of the host's own volume cancel it.
- **Replay lies about entry lag.** The replay keeps the original timestamps, so entry lag is the archive's age, about a month. Shipping lag stays real on replay.
- **Small numbers.** Rules need 50 records a slice and 10 errors for an error rule. The floors come from the archive, where daily volume ranges 220 times.
- **Retired hosts.** A collapse needs a host that was busy in the last day, so a decommissioned machine clears itself after 24 hours.

7.6 The signal projector: one history for every alert

In one sentence: the projector turns every alert and every grade into signals and episodes that are stored, deduplicated and have a start and an end.

Thirty monitors and seventeen detectors write to different indices, and a firing monitor writes a new row every minute. Alertmanager [9] groups and routes well, but it keeps nothing: it forgets an alert five minutes after the last post. Every 30 seconds the projector reads all of it and writes:

- **Signals:** one index for every alert and grade, firing and cleared, named from one catalogue.
- **Episodes:** all signals of one alert name on one entity become one incident with a **start and an end**. It goes open, recovering, resolved. It closes after one healthy window for a hard rule, three for a trend rule or detector. A missing evaluation is never a recovery, so a lane that stops reporting leaves its episode open and marked stale. When half the roster goes silent at once, one fleet incident opens instead of hundreds.
- **Re-posts:** every open episode goes to Alertmanager on every cycle, well inside its five-minute memory.

Table 16: The projector remembers and deduplicates, and Alertmanager decides when a person is told.

	Signal projector	Alertmanager
Memory	30 days, replicated	forgets an alert after 5 minutes
Duplicates	thirty rows about one fault are one episode	repeats whatever it is sent
Start and end of a trouble	on every episode	not kept
Read by	the cockpit and the shifter view	the notification receiver
Notification	none of its own	grouping, silences, routing

Why it needed a watermark. Its sources arrive late and out of order: a grade lands after its window closes, a history row when its alert ends, and the projector itself restarts. So it keeps one **watermark** per source, the last moment it fully processed:

1. Read from the watermark minus a short overlap, 15 minutes for alert history and 3 for grades, so late rows are caught.
2. Move the watermark only after a complete read. A failed cycle leaves it, and the next cycle reads the same window again.

3. Build every signal's identifier from its source row, so reading the overlap twice overwrites and never duplicates.

Who watches the watchers. Every monitor runs inside the cluster, so a dead cluster silences all of them at once, and silence reads as health. Table 17 gives the checks on the detection machinery itself.

Table 17: The watchdogs cover the machinery the other rules depend on, and the two break-glass ones skip the path they report dead.

Check	Fires when	Why it exists
opensearch-unreachable	the projector fails two cluster probes in a row	raised by the projector straight to Alertmanager; the one alert that survives a dead cluster
signal-projector-stale	the projector wrote nothing for 10 minutes	break-glass: posts straight to the receiver, since the projector is the thing that is dead
alertmanager-down	the projector saw Alertmanager unreachable for 5 minutes	break-glass: nobody is receiving pages
trend-rollup-stale	no complete rollup commit for 40 minutes	every trend rule is reading stale rows
trend-entity-cap	a rollup group nears its host cap	hosts past the cap are silently unwatched

A dead projector and a dead cluster at the same moment, a site or network loss, needs an off-site heartbeat. That does not exist yet.

8 Limits and what remains

8.1 Not measured

Every cost figure in Section 5 comes from a laptop. One micro-benchmark on one farm machine gives the conversion factor. None has run.

The rig gave the worker's collector and store four exclusive cores, and the storage tier four more. Sharing them with reconstruction is not measured.

No run measured a burst. The soak plan's per-worker burst of 10,000 to 20,000 records a second matches the farm-wide peak of 9,781, not a worker's busiest second of 78.

The whole stack's cost was measured at 5,000 records a second only. The collector alone was also costed at 1,000 and 20,000. Round 1's 53,000 was read against a test output that always accepts, so the collector's ceiling with a real store is unknown above 50,000.

The dead-collector chain of Section 3.7 is bounded at 90 to 270 seconds by cadence, and no run timed it.

A detector needs a few hundred one-minute windows, three to five hours, before its grades mean anything. The injection harness waits 32 one-minute windows. Neither the real warm-up nor that wait was measured on a live feed.

The semantic path was timed on a laptop with a warm index. The collector's 64-chunk burst ceiling is not measured on a worker.

8.2 Limits found in the design

The stale-projector monitor fires only while one projector heartbeat exists in the last 24 hours, so a day of silence mutes it. The Alertmanager-down monitor reads the projector's own report, so a dead projector silences it too.

No inhibition rule is active, because all 22 causal edges between alerts ship unproven.

Nothing authenticates inside the cluster, so one password and a firewall rule are the whole boundary, on five machines we own.

The live lane carries InfoLogger, the daemon log and the central family only. Process output lines with no severity went central before the fix of Section 5.4, so the live page now shows fewer.

DDS is the largest family during data-taking and the dearest per line. The recipe runs under-sample it at 43,972 lines, 0.1 % of the weight, and the later runs used 1,236,971 lines.

At its 20,000-template ceiling the stamper evicts a template, a rebuilt one restarts its count at one, and the change to keep counting is open.

8.3 What remains, in build order

Nine things remain. Figure 9 shows the three layouts. Staging is two workers and three storage machines. The farm pilot is three workers and three storage containers on one infra machine, and it ran the cluster, the collectors and Dashboards. Whether the stamper, the projector and the shifter view ran on the farm is not recorded. Production, the whole farm with at least three storage machines, two replicas and the bus, is not real.

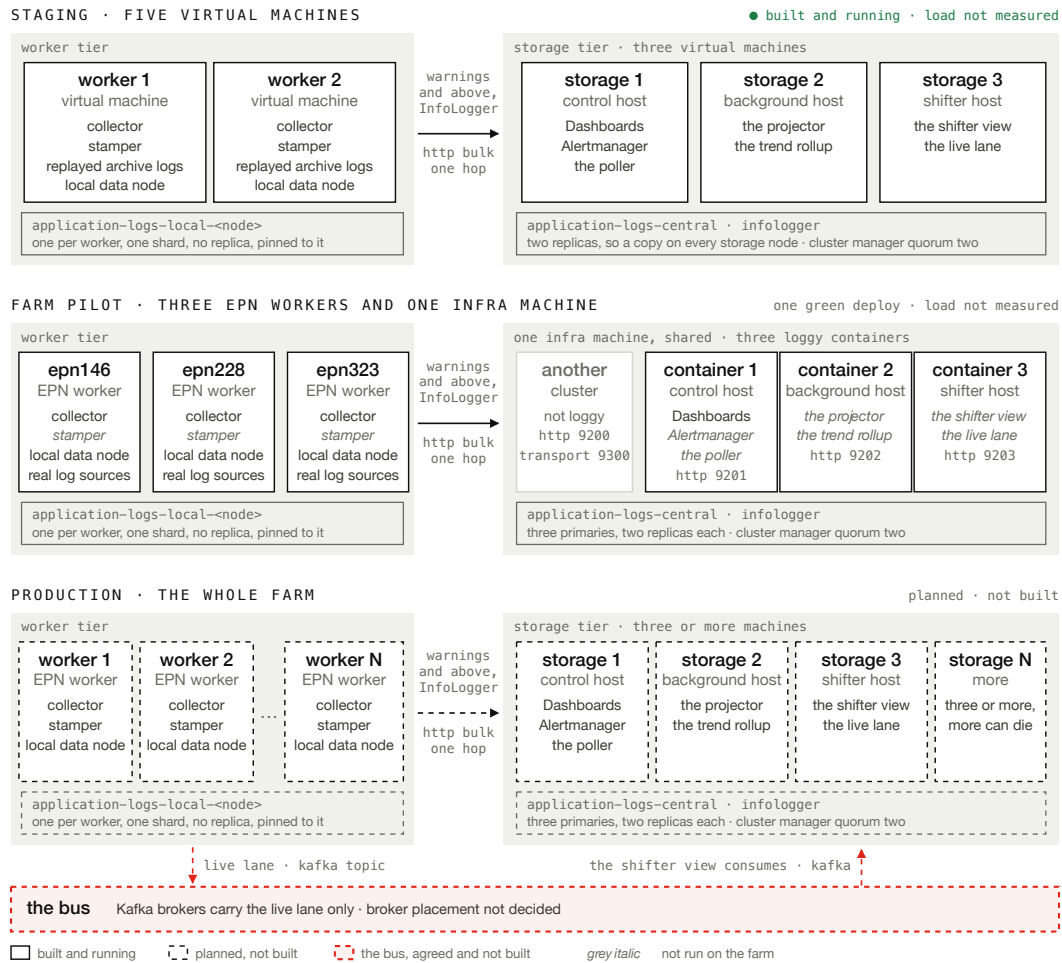


Figure 9: Staging and the farm pilot exist, and production is the same design with a bus and a bigger storage tier.

1. The live-lane bus is agreed and not built: one topic from every collector, the shifter view its one consumer, brokers not placed.
2. The shared Python becomes one versioned package, agreed.
3. The Telegraf decision is open: whether CERN's metrics estate takes over the samples.
4. Catalogue maintenance as ISM policies plus monitors is open. ISM, Index State Management, deletes whole indices by age, so the two counting checks need a home.
5. The operator view beside InfoLogger waits for farm access and a live run.
6. The system journal and per-device process logs are absent from the archive.
7. Detectors on per-template counts. The bucket counts exist, and no detector or trend rule reads them yet. Nearest-neighbour novelty on the text is closed (Section 6).
8. Authentication inside the cluster is required before the farm.
9. Source-owner approval for reading the node logs is open.

9 Conclusion

The logs an EPN writes can stay where they are made and still be searched as one, on four cores. The watching costs a worker nothing it did not push itself.

A median worker carries 23 records a second in the busiest hour of six months of archive. The whole stack on one worker sustains about 42,000 records a second on the laptop rig. The templating pipeline is 31 to 57 % cheaper per family and byte-identical. Dense retrieval with reranking reaches 0.685 nDCG@10 against 0.634 without it.

One limit binds every figure: all come from a laptop, so shapes and rankings transfer to the farm and absolute rates do not.

What exists now is groundwork rather than a demonstration: five staging machines, and one deployment on the farm that passed every check.

Acknowledgements

I thank my supervisors, Lubos Krcal and Federico Ronchetti, for their direction and reviews. The 2025 reference design by Athanasios Papadopoulos [1] gave the project its starting point.

A Upstream roles weighed and not used

Every deployment role is our own, and Table 18 records why each candidate lost.

Table 18: Each upstream role replaces a few stable tasks and hides something the deployment needs.

Component	What it offered	Why it was not used
Store, Dashboards, nginx	Vendor playbooks, two community roles, nginx role	Built around the security plugin we disable. Community roles stale, single-tier. The nginx role owns the site.
Collector	Six community roles	One Debian-only, one without multiline parsers, four only install. The pipeline is the role.
Alertmanager, detection	Prometheus collection role, vendor playbook, Terraform provider	The closest moves the route tree into Jinja. None talks to the detection API. Terraform keeps its own state.
Cockpit metrics, trend rollup	Exporter, Metricbeat, performance analyzer, systemd role, native rollup	Each writes to a store the monitors do not read. A native rollup writes nothing for silence.
Host preparation, template catalogue	Swap, timesync, firewall, kernel roles, ISM, rollover	ISM deletes whole indices by age. Rollover duplicates a catalogue upserted by version.
Replay, faults, ops page, projector, shifter view	None	The software exists only here.

References

- [1] A. Papadopoulos. *Scalable Logging Architecture: expanding InfoLogger into a unified, scalable logging platform*. ALICE Summer Student presentation, CERN, 15 September 2025.
- [2] ALICE O² InfoLogger. <https://github.com/AliceO2Group/InfoLogger>.
- [3] ALICE Collaboration. *Technical Design Report for the Upgrade of the Online–Offline Computing System*. CERN-LHCC-2015-006, ALICE-TDR-019, 2015.
- [4] Fluent Bit documentation. <https://docs.fluentbit.io/>.
- [5] OpenSearch documentation: anomaly detection, alerting, index state management. <https://docs.opensearch.org/>.
- [6] P. He, J. Zhu, Z. Zheng, M. R. Lyu. Drain: an online log parsing approach with fixed depth tree. *IEEE International Conference on Web Services (ICWS)*, 2017.
- [7] Drain3: a streaming implementation of Drain. <https://github.com/logpai/Drain3>.

- [8] S. Guha, N. Mishra, G. Roy, O. Schrijvers. Robust random cut forest based anomaly detection on streams. *International Conference on Machine Learning (ICML)*, 2016.
- [9] Prometheus Alertmanager documentation. <https://prometheus.io/docs/alerting/latest/alertmanager/>.
- [10] Ansible documentation. <https://docs.ansible.com/>.
- [11] Apache Kafka documentation, KRaft mode. <https://kafka.apache.org/documentation/>.
- [12] K. Järvelin, J. Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- [13] Cloud Native Computing Foundation, graduated projects and graduation criteria. <https://www.cncf.io/projects/>.
- [14] A. Techaviseschai, S. Tarnpradab, V. Chibante Barroso, P. Phunchongharn. A real-time semi-supervised log anomaly detection framework for ALICE O² facilities. *Applied Sciences*, 15(11):5901, 2025. doi:10.3390/app15115901.